2 수치, 셀과 구조체 배열

MATLAB Programming

Jin Hee Yoon
Dept. of Artificial Intelligence and Information Technology
Sejong University

Spring, 2026

학습 목표

- 행벡터, 열벡터, 행렬을 생성하는 기본 문법을 익힌다.
- 콜론 연산자, `linspace`, `logspace`의 차이를 이해한다.
- 배열 인덱싱과 부분배열 추출 방법을 학습한다.
- 원소별 연산과 행렬 연산의 차이를 구분한다.
- 특수 행렬, 좌측 나눗셈, 다항식, 셀 배열, 구조체 배열의 기초를 익힌다.

벡터의 기본 생성

행벡터
대괄호 안에 원소를 쓰고, 빈칸 또는 콤마로 구분한다.

$$p = [3 \quad 7 \quad 9]$$

또는

$$p = [3, 7, 9]$$

열벡터
행벡터에 전치 기호 ’를 붙이거나, 세미콜론으로 원소를 구분한다.

$$p = [3, 7, 9]', \qquad g = [3; 7; 9]$$

벡터도 결국은 하나의 행 또는 하나의 열을 가진 특별한 행렬이다.

벡터 생성 예

```
p = [3, 7, 9]
p = [3, 7, 9]'
g = [3; 7; 9]
```

- [3,7,9] : 1×3 행벡터
- [3,7,9]' : 3×1 열벡터
- [3;7;9] : 3×1 열벡터

벡터 연결

서로 같은 방향의 벡터는 연결하여 새로운 벡터를 만들 수 있다.

$$r = [2, 4, 20], \qquad w = [9, -6, 3]$$

$$u = [r, w] = [2, 4, 20, 9, -6, 3]$$

행벡터끼리는 가로로 이어 붙이고, 열벡터끼리는 세로로 이어 붙이는 것이 자연스럽다.

콜론 연산자

콜론 연산자 :는 일정한 간격의 벡터를 매우 쉽게 만든다.

$$x = m : q : n$$

- m: 시작값, q: 간격, n: 끝값
- x=[m:q:n] 도 가능
- q 생략시 간격을 default 로 1로 설정함

예

$$0 : 2 : 8 = [0, 2, 4, 6, 8]$$

$$0 : 2 : 7 = [0, 2, 4, 6]$$

$$-3 : 2 = [-3, -2, -1, 0, 1, 2]$$

콜론 연산자 예

```
x = 0:2:8
y = 0:2:7
z = 5:0.1:8
w = -3:2
```

- 끝값이 간격에 정확히 맞지 않으면 마지막 원소는 끝값보다 작을 수 있다.
- 간격을 생략하면 기본 간격은 1이다.

`linspace`와 `logspace`

linspace(x1,x2,n)

- 구간 $[x_1, x_2]$를 **등간격**으로 나누어 원소 n개를 생성한다.

logspace(a,b,n)

- 10^a부터 10^b까지 **로그 간격**으로 원소 n개를 생성한다.

예

$$\texttt{linspace(5,8,31)} \equiv [5 : 0.1 : 8]$$

$$\texttt{logspace(-1,1,4)} = [0.1,\ 0.4642,\ 2.1544,\ 10]$$

벡터의 길이, 크기, 절대값

벡터 $x = [x_1, x_2, \ldots, x_n]$에 대해:

- **길이(length)**: 원소의 개수

```
length(x)
```

- **크기(magnitude)**: 유클리드 노름

$$\|x\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

```
norm(x)
```

- **절대값(absolute value)**: 각 원소의 절대값

```
abs(x)
```

길이, 크기, 절대값의 예

$$x = [2, -4, 5]$$

- 길이: $\texttt{length(x)} = 3$
- 크기:

$$\texttt{norm(x)} = \sqrt{2^2 + (-4)^2 + 5^2} = 6.7082$$

- 절대값:

$$\texttt{abs(x)} = [2, 4, 5]$$

```
x = [2, -4, 5];
length(x)
norm(x)
abs(x)
```

행렬의 개념과 생성

행렬은 여러 개의 행과 열을 갖는 배열이다.

$$A = [2, 4, 10; 16, 3, 7]$$

$$A = \begin{bmatrix} 2 & 4 & 10 \\ 16 & 3 & 7 \end{bmatrix}$$

- 같은 행의 원소는 빈칸이나 콤마로 구분
- 행과 행 사이는 세미콜론으로 구분

벡터로부터 행렬 만들기

$$a = [1, 3, 5], \qquad b = [7, 9, 11]$$

가로 연결

$$[a\ b] = [1, 3, 5, 7, 9, 11]$$

세로 연결

$$[a; b] = \begin{bmatrix} 1 & 3 & 5 \\ 7 & 9 & 11 \end{bmatrix}$$

연결 방향이 달라지면 결과 배열의 크기도 달라진다.

배열 주소 지정 I

MATLAB에서는 인덱스를 이용해 원소, 행, 열, 부분배열을 선택한다.

- `v(:)` : 벡터의 모든 원소
- `v(2:5)` : 2번째부터 5번째 원소
- `A(:,3)` : 3번째 열 전체
- `A(:,2:5)` : 2번째부터 5번째 열 전체

배열 주소 지정 II

- `A(2:3,1:3)` : 2–3행, 1–3열 부분배열
- `A(:)` : 모든 열을 순서대로 쌓은 열벡터
- `A(end,:)` : 마지막 행
- `A(:,end)` : 마지막 열

`end`를 사용하면 배열 크기를 미리 몰라도 마지막 위치를 편리하게 지정할 수 있다.

부분배열 추출 예

$$B = \begin{bmatrix} 2 & 4 & 10 & 13 \\ 16 & 3 & 7 & 18 \\ 8 & 4 & 9 & 25 \\ 3 & 12 & 15 & 17 \end{bmatrix}$$

$$C = B(2:3, 1:3) = \begin{bmatrix} 16 & 3 & 7 \\ 8 & 4 & 9 \end{bmatrix}$$

```
C = B(2:3,1:3)
```

추가적인 배열 함수 I

명령어	설명
find(A)	0이 아닌 원소의 위치와 값을 찾음
length(A)	벡터면 원소 수, 행렬이면 행과 열 중 큰 차원 길이
max(A)	벡터면 최댓값, 행렬이면 각 열의 최댓값
min(A)	벡터면 최솟값, 행렬이면 각 열의 최솟값

추가적인 배열 함수 II

명령어	설명
$[x, k] = \max(A)$	최댓값과 그 인덱스 반환
$[x, k] = \min(A)$	최솟값과 그 인덱스 반환
$\mathrm{size}(A)$	배열 크기 $[m, n]$ 반환
$\mathrm{sort}(A)$	각 열을 오름차순 정렬
$\mathrm{sum}(A)$	각 열의 합 계산

`size`와 `length`의 차이

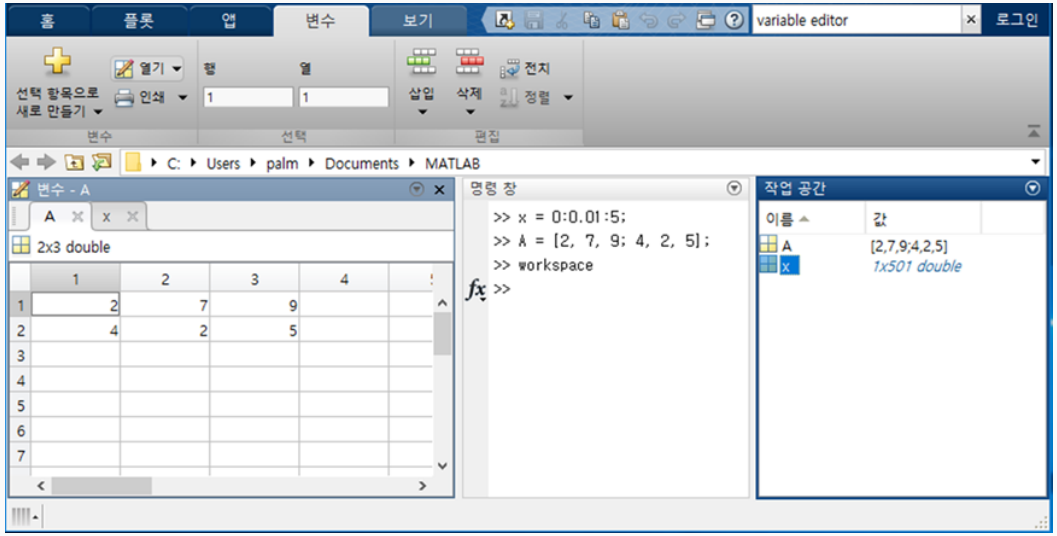
$$A = \begin{bmatrix} 6 & 2 \\ -10 & -5 \\ 3 & 0 \end{bmatrix}$$

- `size(A)` $= [3, 2]$
- `length(A)` $= 3$
- `max(A)` $= [6, 2]$
- `min(A)` $= [-10, -5]$

행렬에서는 `length(A)`가 원소 개수가 아니라 **더 큰 차원의 길이**를 준다는 점에 주의한다.

작업공간 브라우저와 변수 편집기

- Workspace에서 현재 변수 이름과 크기를 확인할 수 있다.
- Variable Editor 에서는 배열을 표 형태로 직접 볼 수 있다.
- 명령창에서 만든 배열을 시각적으로 확인할 때 유용하다.



다차원 배열

3차원 배열은 여러 개의 2차원 행렬을 층처럼 쌓아 만든다.

연결 함수

```
cat(n,A,B,C,...)
```

배열 $A, B, C, \ldots$를 n차원 방향으로 연결한다.

예를 들어 3차원 배열에서는 각 2차원 행렬을 하나의 **페이지 (page)**로 생각할 수 있다.

배열의 덧셈과 뺄셈

같은 크기의 배열끼리는 같은 위치의 원소끼리 더하고 뺀다.

$$A = \begin{bmatrix} 6 & -2 \\ 10 & 3 \end{bmatrix}, \qquad B = \begin{bmatrix} 9 & 8 \\ -12 & 14 \end{bmatrix}$$

$$A + B = \begin{bmatrix} 15 & 6 \\ -2 & 17 \end{bmatrix}$$

덧셈과 뺄셈은 **배열의 크기가 같아야** 정의된다.

스칼라와 배열의 곱

스칼라가 배열에 곱해지면 모든 원소에 같은 값이 곱해진다.

$$A = \begin{bmatrix} 2 & 9 \\ 5 & -7 \end{bmatrix} \quad \Longrightarrow \quad 3A = \begin{bmatrix} 6 & 27 \\ 15 & -21 \end{bmatrix}$$

```
3*A
```

스칼라 곱은 배열 전체를 한 번에 확대하거나 축소하는 효과를 갖는다.

원소별 연산과 행렬 연산

MATLAB에는 두 종류의 곱셈이 있다.

1. 원소별 연산
같은 위치의 원소끼리 계산한다.

$$.*, \quad ./, \quad .\backslash, \quad \ldots$$

2. 행렬 연산
선형대수에서의 행렬곱 개념을 따른다.

$$*, \quad /, \quad \backslash,$$

원소별 연산 기호 정리

기호	의미
`A + B`	배열 덧셈
`A - B`	배열 뺄셈
`A .* B`	원소별 곱셈
`A ./ B`	원소별 오른쪽 나눗셈
`A .\B`	원소별 왼쪽 나눗셈
`A .^ B`	원소별 지수승

원소별 곱셈 예

$$x = [2, 4, -5], \qquad y = [-7, 3, -8]$$

$$z = x.*y = [2(-7),\ 4(3),\ (-5)(-8)] = [-14, 12, 40]$$

```
x = [2,4,-5];
y = [-7,3,-8];
z = x.*y
```

- 원소별 연산은 두 배열의 크기가 같아야 한다.
- 행벡터와 열벡터를 섞으면 크기 불일치 에러가 날 수 있다.

원소별 곱셈과 행렬곱의 차이

$$A = \begin{bmatrix} 11 & 5 \\ 9 & -4 \end{bmatrix}, \qquad B = \begin{bmatrix} -7 & 8 \\ 6 & 2 \end{bmatrix}$$

$$A.*B = \begin{bmatrix} 11(-7) & 5(8) \\ 9(6) & (-4)(2) \end{bmatrix} = \begin{bmatrix} -77 & 40 \\ 54 & -8 \end{bmatrix}$$

`A.*B`는 같은 위치끼리 곱하는 것이고, `A*B`는 전혀 다른 행렬곱이다.

벡터화 함수

`sqrt(x)`, `exp(x)`, `sin(x)` 같은 MATLAB 내장함수는 배열 입력에 자동으로 작용한다.

$$x = [4, 16, 25] \quad \Longrightarrow \quad \texttt{sqrt(x)} = [2, 4, 5]$$

이처럼 배열 전체에 한 번에 적용되는 함수를 **벡터화 함수**라고 한다.

원소별 연산을 함께 써야 하는 경우

배열 x, y에 대해

$$z = e^y \sin x \cos^2 x$$

를 계산하려면 다음과 같이 입력해야 한다.

```
z = exp(y).*sin(x).*(cos(x)).^2;
```

- 함수 자체는 배열에 자동 적용된다.
- 하지만 곱셈, 나눗셈, 지수승은 배열일 때 원소별 기호가 필요하다.

배열 나눗셈과 배열 지수승

원소별 나눗셈

$$x = [8, 12, 15], \quad y = [-2, 6, 5]$$

$$x./y = [8/(-2),\ 12/6,\ 15/5] = [-4, 2, 3]$$

원소별 지수승

$$x = [3, 5, 8] \quad \Longrightarrow \quad x.^3 = [27, 125, 512]$$

$$p = [2, 4, 5] \quad \Longrightarrow \quad 3.^p = [9, 81, 243]$$

행렬-행렬 곱셈

행렬곱 AB가 가능하려면

$$\text{A의 열 수} = \text{B의 행 수}$$

이어야 한다.

$$A = \begin{bmatrix} 6 & -2 \\ 10 & 3 \\ 4 & 7 \end{bmatrix}, \qquad B = \begin{bmatrix} 9 & 8 \\ -5 & 12 \end{bmatrix}$$

그러면

$$A * B = \begin{bmatrix} 64 & 24 \\ 75 & 116 \\ 1 & 116 \end{bmatrix}$$

행렬곱 계산 예

```
A = [6,-2;10,3;4,7];
B = [9,8;-5,12];
A*B
```

- 결과 행렬의 크기는

$$(\text{A의 행 수}) \times (\text{B의 열 수})$$

  이다.
- 위 예에서는 3×2 행렬이 된다.

행렬곱은 일반적으로 교환법칙이 성립하지 않음

보통

$$AB \neq BA$$

이다.

행렬의 곱에서 순서를 바꾸는 것은 매우 흔한 실수이다.

- `A*B`가 가능해도 `B*A`는 불가능할 수 있다.
- 둘 다 가능하더라도 결과는 일반적으로 다르다.

특수한 행렬

영행렬
모든 원소가 0인 행렬

$$0A = A0 = 0$$

단위행렬
대각선 원소만 1이고 나머지는 0인 정방행렬

$$IA = AI = A$$

빈 행렬 []는 영행렬과 다르다.

eye, zeros, ones

명령어	설명
$\mathrm{eye}(n)$	$n \times n$ 단위행렬
$\mathrm{eye}(\mathrm{size}(A))$	A와 같은 크기의 단위행렬
$\mathrm{zeros}(n)$	$n \times n$ 영행렬
$\mathrm{zeros}(m, n)$	$m \times n$ 영행렬
$\mathrm{ones}(m, n)$	$m \times n$ 모든 원소가 1인 행렬

행렬 좌측 나눗셈과 선형연립방정식

연립방정식

$$6x + 12y + 4z = 70$$

$$7x - 2y + 3z = 5$$

$$2x + 8y - 9z = 64$$

를

$$A\mathbf{x} = B$$

꼴로 쓰면

$$A = \begin{bmatrix} 6 & 12 & 4 \\ 7 & -2 & 3 \\ 2 & 8 & -9 \end{bmatrix}, \qquad B = \begin{bmatrix} 70 \\ 5 \\ 64 \end{bmatrix}$$

이다.

좌측 나눗셈 예

해는 MATLAB에서 다음처럼 구한다.

```
A = [6,12,4;7,-2,3;2,8,-9];
B = [70;5;64];
Solution = A\B
```

결과:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 3 \\ 5 \\ -2 \end{bmatrix}$$

즉 $x = 3,\ y = 5,\ z = -2$이다.

다항식 계수와 근

`poly(r)`
근이 r인 다항식의 계수를 반환한다.

`roots(a)`
계수 배열 a로 표현된 다항식의 근을 반환한다.

$$\texttt{poly([-2,-5])} = [1, 7, 10]$$

$$\texttt{roots([2,14,20])} = \begin{bmatrix} -2 \\ -5 \end{bmatrix}$$

다항식 곱과 나눗셈

- `conv(a,b)` : 두 다항식의 곱
- `[q,r]=deconv(num,den)` : 다항식 나눗셈의 몫과 나머지

계수 배열을 이용하면 다항식도 MATLAB에서 일반 배열처럼 다룰 수 있다.

다항식 곱과 나눗셈 예

```
a = [9,-5,3,7];
b = [6,-1,2];

product = conv(a,b)

[quotient, remainder] = deconv(a,b)
```

- conv는 계수 배열의 곱을 계산한다.
- deconv는 몫 다항식과 나머지 다항식을 분리한다.

다항식 값 계산과 그래프

`polyval(a,x)`는 계수 배열 a로 정의된 다항식의 값을 입력 x에서 계산한다.

예를 들어

$$f(x) = 9x^3 - 5x^2 + 3x + 7$$

의 그래프를 $-2 \leq x \leq 5$에서 그리려면,

$$a = [9, -5, 3, 7]$$

를 사용한다.

다항식 그래프 예

```
a = [9,-5,3,7];
x = -2:0.01:5;
f = polyval(a,x);

plot(x,f)
xlabel('x')
ylabel('f(x)')
grid on
```

- 먼저 입력 벡터 x를 만든다.
- polyval로 함수값을 계산한다.
- plot으로 시각화한다.

셀 배열

셀 배열은 서로 다른 자료형이나 서로 크기가 다른 데이터를 한 배열 안에 저장할 수 있게 해준다.

명령어	설명
$\mathrm{cell}(n)$	빈 $n \times n$ 셀 배열 생성
$\mathrm{cell}(n, m)$	빈 $n \times m$ 셀 배열 생성
$\mathrm{celldisp}(C)$	셀 배열 내용 출력
$\mathrm{cellplot}(C)$	셀 배열 구조를 그림으로 표시

구조체 배열

구조체 배열은 **필드(field)**를 이용해 서로 관련된 데이터를 이름과 함께 저장한다.

예를 들어 학생 정보를 저장할 때,

```
name, id, score
```

처럼 의미 있는 필드명을 둘 수 있다.

배열이 위치 기반 저장이라면, 구조체는 **이름 기반 저장**에 가깝다.

구조체 배열의 데이터 구조

- 구조체는 여러 필드를 가진다.
- 각 필드는 하나의 속성을 의미한다.
- 실제 데이터 관리에서 매우 자주 사용된다.

학생, 센서, 실험 데이터처럼 여러 속성을 함께 묶어야 할 때 구조체가 유용하다.

구조체 함수

명령어	설명
$\mathrm{fieldnames}(S)$	구조체의 필드명 반환
$\mathrm{isfield}(S, 'field')$	특정 필드 존재 여부 확인
$\mathrm{isstruct}(S)$	구조체인지 판별
$\mathrm{rmfield}(S, 'field')$	특정 필드 삭제
$\mathrm{struct}('f1',' v1', \ldots)$	구조체 생성

구조체 생성의 예

```
student = struct( ...
    'name', 'Kim', ...
    'id', 2026001, ...
    'score', 95 );
```

- `student.name`
- `student.id`
- `student.score`

처럼 각 필드에 접근할 수 있다.

실습 문제

1. `[1,3,5]`와 `[1;3;5]`의 차이를 설명하라.
2. `0:0.5:5`와 `linspace(0,5,11)`를 비교하라.
3. 주어진 행렬에서 마지막 행과 마지막 열을 추출하라.
4. `A.*B`와 `A*B`의 차이를 예로 설명하라.
5. `A\B`를 이용하여 간단한 연립방정식을 풀어보라.
6. `polyval`을 이용해 다항식 그래프를 그려보라.